

Day 3 — Analog & Data: Full Instructor Teaching Material (Technical Edition)

Total session time: 210 minutes (3.5 hrs). Each section is sized to its time budget with talking points, board work, worked math, register-level detail, and explicit framing lines.

1. Analog Inputs/Outputs — 40 min

1a. How Analog Signals Work — 8 min

Two industry-standard analog signal types, with electrical specifics:

- **4-20mA current loop** — a current source, not a voltage source. Advantage worth stating explicitly: current loops are far less susceptible to voltage drop over long wire runs than voltage signals, which is why they dominate in industrial field wiring over distances of tens to hundreds of meters. In a current loop, the same current flows through every device in the series circuit regardless of wire resistance, up to the loop's compliance voltage limit — this is the actual electrical reason distance doesn't degrade the signal the way it does for a voltage-based signal.
- **0-10V voltage signal** — simpler to wire (no loop-powering considerations, and doesn't require a series circuit topology) but degrades over long cable runs due to resistive voltage drop ($V = IR$ losses across the cable itself) and is more susceptible to induced electrical noise from nearby motor drives, VFDs, and other electrically noisy equipment common on a factory floor.

Why 4-20mA rather than 0-20mA — the live-zero justification, stated precisely: A reading of exactly 0 is inherently ambiguous in a 0-based range: is the process variable genuinely at its minimum, or has the loop failed (broken wire, disconnected transmitter, dead power supply to a 2-wire transmitter)? By offsetting the live range to start at 4mA, a measured 0mA can only be interpreted as a fault condition — it is physically outside the valid signal range. This is formally called a "live zero" and is a deliberate design decision in the 4-20mA standard (ISA/IEC standard practice), not an arbitrary offset.

2-wire vs. 4-wire transmitters, worth a brief mention since it directly explains why 4mA specifically (not some other nonzero value) was chosen historically: many industrial transmitters are "2-wire" or "loop-powered" devices — the same two wires carrying the 4-20mA signal also supply the transmitter's own operating power, drawn from that same current loop. 4mA was chosen as a value high enough to reliably power typical transmitter electronics of the era while still leaving a full 16mA of usable signal range (4-20mA) above it.

1b. Raw ADC Counts vs. Engineering Units — 8 min

ADC resolution, stated with actual bit math:

- A 10-bit ADC produces raw values 0-1023 ($2^{10} - 1 = 1023$)
- A 12-bit ADC produces raw values 0-4095 ($2^{12} - 1 = 4095$)
- Some PLC platforms (Siemens S7 analog modules, for example) use a normalized raw range of 0-27648, chosen for specific internal scaling reasons on that platform rather than a pure power-of-two ADC width

State explicitly: the ADC's bit resolution determines the smallest detectable change in the input signal. For

a 10-bit ADC across a 0–100°C range: resolution = 100°C / 1023 counts $\approx$ **0.0978°C per count** — meaning the system genuinely cannot distinguish two temperatures closer together than roughly 0.1°C, regardless of sensor accuracy. Worth stating that this is a hardware resolution limit distinct from sensor accuracy — the two are often conflated by students. A sensor rated to $\pm 0.5^\circ\text{C}$ accuracy paired with a 12-bit ADC (resolution $\approx 0.024^\circ\text{C}/\text{count}$ over the same range) is still only as good as the sensor's $\pm 0.5^\circ\text{C}$ — the finer ADC resolution doesn't improve the physical measurement's real-world accuracy, it just avoids the ADC itself being the bottleneck.

Sampling rate, worth a brief mention even though it's not the day's focus: an analog input module samples the incoming signal at some fixed rate (often tied to or independent of the PLC scan cycle depending on module architecture), and for genuinely fast-changing analog signals (rare in typical temperature/pressure applications, common in some vibration or flow-metering applications), sample rate can itself become a limiting factor distinct from ADC bit resolution — flag as a "beyond today's scope" aside rather than a focus area.

Pipeline to keep visible on the board:

```
Physical signal (4-20mA) --> ADC --> Raw count (0-1023, INT) --> Scaling --> Engineering value (REAL)
```

1c. Scaling Formula Walkthrough — 12 min

Formula:

$$EU = ((\text{Raw} - \text{RawMin}) / (\text{RawMax} - \text{RawMin})) \times (\text{EUMax} - \text{EUMin}) + \text{EUMin}$$

Worked example 1 — temperature, 0-1023 raw representing 0-100°C:

```
Raw = 512
EU = ((512 - 0) / (1023 - 0)) * (100 - 0) + 0
    = (512 / 1023) * 100
    = 0.500489... * 100
    = 50.0489°C (round for display, keep full precision internally)
```

Worked example 2 — pressure, 0-27648 raw representing -10 to 50 PSI (non-zero EUMin, forces the +EUMin term to actually matter):

```
Raw = 13824
EU = ((13824 - 0) / (27648 - 0)) * (50 - (-10)) + (-10)
    = 0.5 * 60 + (-10)
    = 30 - 10
    = 20 PSI
```

Worked example 3 — a case with a non-zero RawMin, to show the formula handles that term too (e.g., a signal representing 4-20mA already converted by the module to a raw range of 6400-32000 rather than starting at 0, representing 0-200°C):

```
Raw = 19200
EU = ((19200 - 6400) / (32000 - 6400)) * (200 - 0) + 0
    = (12800 / 25600) * 200
    = 0.5 * 200
    = 100°C
```

State explicitly: this third example matters because students will encounter modules where the raw range genuinely doesn't start at 0 — for instance, a module that reports raw counts corresponding directly to the 4-20mA range rather than a 0-20mA equivalent — and the formula must be applied with the *actual* RawMin/RawMax for that specific module, not assumed to always start at zero.

Built-in instruction breakdown (IEC-style two-step, common on Siemens/CODESYS platforms):

```
NORM_X(IN := RawValue, MIN := 0, MAX := 1023);      // output: REAL, 0.0-1.0
normalized := NORM_X.OUT;
SCALE_X(IN := normalized, MIN := 0.0, MAX := 100.0); // output: REAL, engineering units
ScaledTemp := SCALE_X.OUT;
```

Or single-instruction form (Allen-Bradley SCL / SCALE_X combined instruction):

```
SCL(Source := RawADC, InputMin := 0, InputMax := 1023,
    ScaledMin := 0.0, ScaledMax := 100.0, Dest := ScaledTemp);
```

State explicitly: the built-in instruction performs exactly the math above internally — walking through it by hand first is what prevents students from treating SCL / SCALE_X as an unexplainable black box, which matters the first time a scaling parameter is entered wrong and the output silently reads incorrect values with no error thrown. Concretely: if InputMax were mistakenly entered as 4095 (a 12-bit range) while the module is actually a 10-bit device producing raw values up to 1023, every scaled value would read roughly 4x too low, with the program compiling and running with no fault at all — this exact class of silent, undetected error is why hand-deriving the formula matters pedagogically even though the instruction is used in practice.

1d. Project: Scale a Simulated Analog Input — 12 min

Task: simulated raw input (0-1023) → SCL / SCALE_X → REAL tag ScaledTemp (0-100°C) → display.

Check while circulating: is ScaledTemp declared REAL (32-bit IEEE 754 floating point), not INT? Flag now, full explanation in Block 4. A second check worth making while circulating: did students correctly enter InputMin / InputMax matching the *actual* simulated raw range (0-1023), or did they guess a different bit-width range from memory — ask them to state where they got the 0-1023 figure from (the ADC bit-math in 1b) to confirm the connection between blocks is being made rather than the numbers being copied without understanding.

2. Arithmetic Instructions — 20 min

2a. ADD/SUB, MUL/DIV — 14 min

Instructions operate on the Block 1 REAL register directly — continuity, not a new dataset.

Instruction syntax (IEC ST form, most portable across platforms):

```
Sum      := ScaledTemp1 + ScaledTemp2;  // ADD
Deviation:= ScaledTemp - Setpoint;      // SUB
Scaled_F := ScaledTemp * 1.8 + 32.0;    // MUL/ADD combined - °C to °F conversion
Average  := Sum / Count;               // DIV
```

Note the °C→°F conversion as a genuinely practical MUL example worth using live: $(^{\circ}\text{C} \times 1.8) + 32 =$

°F. Work one instance numerically on the board using the Block 1c result: $50.0489^{\circ}\text{C} \times 1.8 + 32 = 90.088 + 32 = 122.088^{\circ}\text{F}$, so students see the same value carried through a second transformation rather than treating each block's example as disconnected.

Ladder equivalent, for the ADD/DIV average operation, since students will be building this in ladder form:

```
|-----[ADD ScaledTemp1, ScaledTemp2, Sum]--|  
  
|-----[DIV Sum, 2.0, Average]--|
```

Division-by-zero guard — the critical technical detail, stated as a hard rule: If `Count` can legitimately be 0 (e.g., averaging over a rolling window that hasn't filled yet), the DIV instruction must be guarded:

```
IF Count > 0 THEN  
    Average := Sum / Count;  
ELSE  
    Average := 0.0; // or hold last valid value, or flag a fault bit  
END_IF
```

Platform-specific note worth mentioning: some PLCs fault the entire program (major fault, CPU goes to STOP/FAULT) on an unguarded divide-by-zero at runtime; others silently return an error code (e.g., NaN or Inf for REAL division) without halting. State explicitly this is platform-dependent and must be checked, not assumed — flag it as directly relevant to more complex Day 4 arithmetic logic.

A worked concrete fault scenario worth narrating, not just stating abstractly: imagine a rung computing `Average := Sum / SensorCount`, where `SensorCount` is itself computed elsewhere in the program as "number of currently-online sensors." If every sensor in a group happens to go offline simultaneously (a genuinely plausible field scenario — e.g., a shared power supply to a sensor bank fails), `SensorCount` becomes 0 at runtime, and without a guard, this single rung can take down the entire PLC's scan on a platform that major-faults on divide-by-zero — meaning a sensor wiring fault, that maybe shouldn't be catastrophic on its own, cascades into total loss of control. This is exactly the kind of failure mode a guard condition exists to prevent, and it's worth walking through as a story, not just a rule.

2b. Project: Average Two Analog Readings — 6 min

```
Sum      := ScaledTemp1 + ScaledTemp2;  
Average := Sum / 2.0; // note: 2.0 not 2, forces REAL division not truncating INT division on some
```

Flag explicitly: on platforms where the literal `2` defaults to INT and REAL/INT mixed arithmetic doesn't auto-promote, writing `Sum / 2` instead of `Sum / 2.0` can silently produce integer division behavior depending on operand typing rules — worth checking against the specific platform in use. Have students deliberately try both forms if the lab platform allows quick testing, and compare the two results directly (e.g., averaging 50.05 and 51.05 — REAL division correctly gives 50.55, while a naive INT-truncating path could give a visibly wrong result depending on how the platform handles the mixed-type expression) so the difference is observed rather than just described.

3. Comparison Instructions — 20 min

3a. GT/LT/EQ Concept — 8 min

Boolean-output instructions, IEC ST form:

```
HighAlarm := ScaledTemp > 50.0;    // GT
LowAlarm  := ScaledTemp < 10.0;    // LT
AtSetpoint:= ScaledTemp = 50.0;    // EQ - flagged here, cautioned properly in the Day Project
```

Ladder equivalent, since these will be built as ladder rungs in the projects:

```
|-- [ ScaledTemp GT 50.0 ]----- ( HighAlarm )--|
|-- [ ScaledTemp LT 10.0 ]----- ( LowAlarm )--|
```

GE/LE variants, worth naming alongside GT/LT since the choice between them matters for boundary behavior (this directly seeds the boundary discussion in the Day Project's edge-case review):

```
GE: ScaledTemp >= 50.0    // includes exactly 50.0 in the "true" case
LE: ScaledTemp <= 50.0    // includes exactly 50.0 in the "true" case
```

Ask the class explicitly: for a high-temperature safety alarm, would you rather use GT (alarm at anything strictly above the setpoint) or GE (alarm at the setpoint itself and above)? There's a legitimate safety-margin argument for GE being the more conservative choice in a real safety-related alarm — worth surfacing as a design decision rather than treating GT as the unquestioned default.

Frame explicitly as the data→logic bridge: everything computed as a REAL number in Blocks 1-2 only becomes actionable once passed through a comparison producing a BOOL.

3b. Project: Trigger Output if Temperature Exceeds 50°C — 12 min

```
|-- [ ScaledTemp GT 50.0 ]----- ( HighTempAlarm )--|
```

Full pipeline now assembled and worth diagramming explicitly on the board:

```
Raw (INT, 0-1023) --SCL--> ScaledTemp (REAL, °C) --GT 50.0--> HighTempAlarm (BOOL)
```

While circulating, have students test their rung against three specific raw input values, not just one arbitrary test: a value that should clearly read below 50°C, a value that should clearly read above it, and a value very close to the boundary (e.g., raw $\approx$ 511 or 512, corresponding to right around 50°C per the Block 1c example) — this last test is what actually exercises the GT-vs-GE boundary question raised in 3a, rather than leaving it purely theoretical.

4. Data Types — 20 min

4a. BOOL vs. INT vs. REAL — 10 min

Precise sizes and ranges, worth putting on the board as a table:

--	--	--	--

Type	Size	Range	Typical use
BOOL	1 bit	0 or 1	Digital I/O, logic states, alarm flags
INT	16-bit signed	−32,768 to 32,767	Raw ADC counts (fits within 10/12-bit range), small counters
DINT	32-bit signed	−2,147,483,648 to 2,147,483,647	Larger counters, some raw ranges (e.g., 0-27648 fits INT but larger accumulators need DINT)
REAL	32-bit IEEE 754 floating point	$\sim \pm 3.4 \times 10^{38}$, ~ 7 significant decimal digits	Scaled engineering values, anything needing fractional precision

Note explicitly: REAL's ~ 7 significant digit precision is itself a limit — it's not infinite precision, which is directly why the EQ-comparison caution in Block 3a/5-4 exists: two REAL values that are mathematically "equal" can differ in their least significant bits due to floating-point representation error.

A brief, concrete illustration of why floating point representation error happens at all, worth including since it demystifies what otherwise looks like an arbitrary warning: IEEE 754 REAL values are stored in binary, and many decimal fractions that look "clean" in base 10 (like 0.1) cannot be represented exactly in binary floating point, the same way $1/3$ cannot be represented exactly as a finite decimal. A value computed as "50.0" through a chain of scaling and division operations may actually be stored internally as something like 49.999998 or 50.000002 — visually identical when displayed rounded to 2 decimal places, but not bit-identical, which is exactly what breaks an EQ comparison.

INT overflow, worth a concrete example since it's directly relevant to raw ADC storage decisions: if a raw analog value were mistakenly stored in a data type too small for its range — e.g., attempting to store the Siemens-style 0-27648 raw range in an 8-bit type (range 0-255, if such a type were used) — the stored value would wrap around or truncate incorrectly. This isn't a live concern for a 0-1023 range and a 16-bit INT (1023 fits comfortably within INT's 32,767 max), but it's worth stating as a general principle: always confirm the chosen data type's range comfortably exceeds the actual data range expected, with margin, not just barely fits.

4b. Brief Mention of Arrays — 5 min

```
Temp : ARRAY[0..3] OF REAL;    // Temp[0] through Temp[3], four sensors, one tag
```

Memory layout note: an array of 4 REALs occupies 4×4 bytes = 16 bytes of contiguous memory, addressed by a single base tag with an index offset — contrast this with declaring Temp0, Temp1, Temp2, Temp3 as four entirely separate tags with no structural relationship to each other.

A brief mention of why the index-based addressing matters practically, beyond just tidiness: with an array, a loop construct (e.g., a FOR loop in structured text, iterating Temp[i] for i := 0 TO 3) can process all four sensor values with one piece of logic instead of four separately-written, nearly-identical rungs — genuinely useful once sensor counts grow beyond a handful, though full loop constructs are beyond this course's scope, worth a one-line forward-pointer rather than a full explanation.

4c. Why REAL Matters — 5 min

Concrete truncation math, using the exact Block 1c value:

```
As REAL: 50.0489°C    (full precision retained)
As INT:  50°C          (fractional part discarded entirely — not rounded, truncated)
```

State plainly: this failure mode produces no error, no warning — the program compiles and runs normally, and the numeric error compounds silently through every downstream calculation that uses this value (e.g., the averaging in Block 2, the comparison in Block 3). Walk through the compounding concretely: if `ScaledTemp` were INT and held at 50 (truncated from 50.0489), and this value fed into the °F conversion from Block 2a, the result would be $50 \times 1.8 + 32 = 122.0^\circ\text{F}$ instead of the correct 122.088°F — a small individual error, but one that would grow across a longer chain of dependent calculations, and one that's completely invisible unless someone happens to check against an independently-verified reference value.

5. Day Project — Temperature Monitoring System — 110 min

Goal, kept on the board: analog sensor → scale to °C (REAL) → compare against setpoint → trigger fan/alarm output → log alarm event on first transition to true.

1. Design — 10 min

```
RawADC (INT) --SCL--> ScaledTemp (REAL) --GT Setpoint--> HighTempAlarm (BOOL) --R_TRIG--> LogEntryTr
```

Note the `R_TRIG` addition here explicitly — this is what converts a continuously-true alarm condition into a single-shot "log this once" trigger, directly reusing the edge-detection concept from Day 2, Block 2c. State this callback explicitly: "This is exactly the same `R_TRIG`/edge-detection instruction from yesterday, applied to a new problem — logging an event exactly once when it first occurs, not once per scan for as long as it stays true."

Have students sketch this full five-stage pipeline on paper, labeling each arrow with the actual data type crossing it (INT, REAL, BOOL, BOOL-pulse) — this reinforces the data-type discussion from Block 4 by making students explicitly track type transitions through the pipeline rather than treating "the pipeline" as one undifferentiated flow.

2. Build Scaling + Arithmetic — 30 min

Implement Block 1's `SCL` instruction with a REAL destination tag. Hold this requirement firmly. While circulating, verify students are entering the InputMin/InputMax values that actually match the platform's simulated raw range, not a value copied from the board example without checking — a good verification question: "how do you know your InputMax should be 1023 and not 4095?" and expect an answer rooted in ADC bit-width (Block 1b), not "because that's what was on the slide."

3. Add Comparison + Alarm/Event Log — 30 min

```
--[ ScaledTemp GT Setpoint ]----- ( HighTempAlarm )--|
--[ HighTempAlarm ]--[ONS]----- ( LogTrigger )--|
--[ LogTrigger ]-----[MOV CurrentTime, AlarmLogTimestamp]--|
                        [MOV ScaledTemp, AlarmLogValue]--|
```

State explicitly why the `ONS / R_TRIG` is mandatory here and not optional: without it, `LogTrigger` would remain true for every scan the temperature stays above setpoint — potentially thousands of scans per second — flooding a log with duplicate entries instead of the single "alarm became active" event actually

wanted. Frame this as a light preview of the SCADA alarm/event concept formalized on Day 5.

A worth-raising extension question for stronger students, not required for the base project:

what should happen when the alarm condition clears (temperature drops back below setpoint) — should that also generate a log entry (an "alarm cleared" event, distinct from "alarm active")? If yes, this requires an `F_TRIG` (falling-edge trigger) on the same `HighTempAlarm` bit, producing a second, separate log-trigger pulse. Raise this as an open design question rather than assigning it as required work, since it previews exactly the kind of two-state (active/cleared) event logging that becomes a formal requirement in real SCADA alarm systems on Day 5.

4. Test & Debug — 30 min

EQ-comparison bug, explained precisely at the bit level: A comparison written as `ScaledTemp = 50.0` (EQ) can fail to trigger even when the temperature has clearly crossed 50°C, because the computed REAL value might actually be stored as `50.00000012` or `49.99999988` due to floating-point rounding accumulated through the scaling and arithmetic chain — and that value is not bit-for-bit identical to the literal `50.0`. Always use `GT / GE` (or `LT / LE`) for REAL comparisons against a threshold, never exact `EQ`.

A second bug worth explicitly checking for, tying back to Block 2's division guard: if any student's build includes an averaging or division step feeding into the alarm comparison, verify a divide-by-zero guard is present, per Block 2a's discussion — this is a natural point in the project where that earlier lesson either gets correctly applied or gets forgotten, worth a specific circulating check rather than assuming it was retained from an hour and a half earlier.

A third bug worth watching for, specific to the log-trigger rung: students sometimes wire `ONS / R_TRIG` directly on `ScaledTemp GT Setpoint`'s *contact* rather than on a separate intermediate `HighTempAlarm` coil first — functionally this can still work on some platforms, but it conflates the alarm state itself (which should persist as long as the condition is true, since other logic or a display might need to read it continuously) with the one-shot logging trigger (which should not persist). Worth having students articulate why these need to be two separate bits, not collapsed into one, if the distinction isn't obvious from their own build.

5. Review Edge Cases — 10 min

Boundary behavior: clarify whether the implemented comparison is `GT` (strictly greater than 50.0, so exactly 50.0 does NOT alarm) or `GE` (50.0 exactly does alarm) — have students state which their own rung implements and whether that matches the intended process safety margin, directly following up on the GT-vs-GE design question raised back in Block 3a.

Hysteresis, defined with actual numbers:

```
IF ScaledTemp > 50.0 THEN HighTempAlarm := TRUE;    // turns ON above 50.0
IF ScaledTemp < 48.0 THEN HighTempAlarm := FALSE;    // turns OFF only below 48.0
// between 48.0 and 50.0: alarm holds its last state (dead-band)
```

State explicitly: without this dead-band, a reading oscillating between 49.9°C and 50.1°C due to sensor noise would chatter the alarm output on and off every scan it crosses the boundary — a 2°C dead-band (or whatever margin suits the process) prevents that. Conceptual only for today, not required in the build. Note the implementation subtlety worth mentioning even at concept-level: this pattern requires

`HighTempAlarm` to be a *retained* bit across scans (its value in the ELSE-implicit case, i.e. neither condition being met, must simply hold its prior state) rather than a coil driven directly and unconditionally by a single comparison each scan — a small but important structural difference from every comparison rung built so far today, worth flagging even without building it.

If finished early:

Low-temp alarm, direct reuse of the same pattern with LT:

```
| -- [ ScaledTemp LT 10.0 ] ----- ( LowTempAlarm ) -- |
```

Moving average filter (3-sample), shown with actual array indexing:

```
Samples[2] := Samples[1];  
Samples[1] := Samples[0];  
Samples[0] := ScaledTemp;  
FilteredTemp := (Samples[0] + Samples[1] + Samples[2]) / 3.0;
```

State explicitly: this is a simple FIFO shift register pattern (three REAL slots, oldest dropped each scan) — a genuinely common real-world technique for smoothing noisy analog input before it reaches any alarm comparison. Note the ordering dependency explicitly: the shift operations (`Samples[2] := Samples[1]` , etc.) must execute *before* the new value is written into `Samples[0]` , and the array itself directly reuses the array concept introduced in Block 4b — worth pointing out that this is the first place today an array is used for something beyond a passing definition, giving the earlier 5-minute mention concrete payoff.